Chapter 9. Scripting, Part 2

SRC: slides TI

9.1 Adding Logic to Scripts

9.2 Scheduling a Script

Pointers to more training material

- LinuxFun from chapter 23 onwards
- The Linux Command Line from chapter 24 onwards
- LPIC Linux Essentials 3.3
- LPIC 102
- Bash Pitfalls

Exit Codes

Commands (including the scripts and shell functions we write) issue a value to the system when they terminate, called an exit status. This value, which is an integer in the range of 0 to 255, indicates the success or failure of the command's execution.

By convention, a value of zero indicates success and any other value indicates failure. The shell provides a parameter that we can use to examine a command's exit status: `$?`.

In programming languages the "exit code" is returned as return value from main() or as value from an exit() call, e.g. `System.exit(exit_code);` in Java.

In bash script an exit status can be returned with

- `exit 0` to signal success and
- `exit 1` or any value between 1 and 255 to signal failure.

By the way, `exit` is another bash command that is built into the bash shell: there is no `/usr/bin/exit` or other. And no docs via `man 1 exit` but rather via `help exit`.

```
ls
echo $?
find / -name "blablah" 2>/dev/null
echo $?
sgddfg
echo $?
true
echo $?
```

```
false
echo $?
```

Conditional execution of commands

You can execute commands provided that another command has a specific exit status:

- `comm1 && comm2`
  Execute `comm2` only if `comm1` succeeds
  Example: `mkdir t && echo "dir created"`
- `comm1 || comm2`
  Execute `comm2` only if `comm1` does not succeed
  Example: `mkdir t || echo "dir already existed"`

```
#!/bin/bash

today=$(date +%Y%m%d)
backup_dir=/tmp
backup_file=backup${today}.tgz
source_dir="/root"
logfile=/var/log/backup.log

touch /root/test 2>/dev/null || { echo "Execute as root!" 1>&2; exit 1; }
echo "Backing up to ${backup_file}..."
tar -zcf "${backup_dir}/${backup_file" "$source_dir" "${@}" 2>/dev/null
sync
echo "${today}: backup successful" >> "$logfile"
echo "Done."
```

By trying to create a file in the home directory of root (`\root`), we check if the script is being executed as the root user (with e.g. `sudo`). Is this a good way-of-working?

Conditions

The if statement in bash works with the exit-codes of commands:

```
if cp source dest 2>/dev/null
then
  echo "succeeded"
else
  echo "failed"
fi
```

Note how `if` , `then` , `else` and `fi` need to be on separate lines.
To put on a single line:

```
`if cp source dest 2>/dev/null; then echo "succeeded"; else echo "failed";
fi`
```

Different forms of the ´if` statement:

```
if ...      if ...      if ...
then        then        then
  ...         ...         ...
fi          else        else
              ...         ...
            fi          elif
                        then
                          ...
                        else
                          ...
                        fi
```

Example:

```
#!/bin/bash
if grep -F "$1" /etc/passwd 2>/dev/null 1>/dev/null
then
  echo "user $1 exists"
elif grep -iF "$1" /etc/passwd 2>/dev/null 1>/dev/null
then
  echo "user $1 exists approximately..."
else
  echo "user $1 does not exist"
fi
```

Test command

The `test` command checks a certain condition and gives an exit-status 0 if the condition is true.

```
test -f /etc/passwd
echo $?
test "$USER" = "jan"
echo $?
test 5 -gt 7
echo $?
```

The `test` command is the key to more advanced conditional statements:

- test command

```
if test "$USER" != "jan"
then echo "not equal to jan"
else echo "equal to jan"
fi
```

- alternative syntax with `[`

```
if [ "$USER" != "jan" ]
then echo "not equal to jan"
else echo "equal to jan"
fi
```

By the way, `[` is a "real" command! Just try `ls /usr/bin/[` or `which [`.
And `man '['` brings you to the man page of the `test` command.

The [`test`] command has tons of options. See man page or https://en.wikipedia.org/wiki/Test_(Unix).

Test files

`[ -f filename ]` : test if filename is a regular file
`[ -d filename ]` : test if filename is a directory
`[ -e filename ]` : test if filename exists
`[ -r file ]` : test if file is readable
`[ -w file ]` : test if file is writable
`[ -x file ]` : test if file is executable
`[ file1 -nt file2 ]` : test if file1 is newer than file2
`[ file1 -ot file2 ]` : test if file1 is older than file2

Test strings

`[ string1 = string2 ]` : test if string1 is equal to string2
`[ string1 != string2 ]` : test if string1 is different from string2
`[ -n string ]` : test if string is not empty
`[ -z string ]` : test if string is empty

Test integers

`[ number1 -lt number2 ]` : number1 < number2
`[ number1 -le number2 ]` : number1 <= number2
`[ number1 -eq number2 ]` : number1 == number2
`[ number1 -gt number2 ]` : number1 > number2
`[ number1 -ge number2 ]` : number1 >= number2
`[ number1 -ne number2 ]` : number1 != number2

Test: boolean operators

`[ ! $1 -eq 5 ]` : not-operator
`[ $1 -eq 5 -a $2 -eq 6 ]` : AND
`[ $1 -eq 5 -o $2 -eq 6]` : OR
`[ $1 -eq 5 ] && [ $2 -eq 6 ]` : AND
`[ $1 -eq 5 ] || [ $2 -eq 6]` : OR
`[ $1 -eq 5 -a \( $2 -eq 6 -o $3 -eq 7 \) ]` : parentheses

Improving the backup script

To determine if a Bash script has the necessary root privileges, the Effective User ID can be checked for the value 0. With either

- the `id -u` command or
- the value of the `$EUID` environment variable.

Note: checking the output of the `whoami` is considered a "dirty trick".
Just like creating a (temporary) file in the `/root` home folder of user `root`.

```
#!/bin/bash

today=$(date +%Y%m%d)
backup_dir=/tmp
backup_file=backup${today}.tgz
source_dir="/root"
logfile=/var/log/backup.log

[ `id -u` -eq 0 ] || { echo "Execute as root!" 1>&2; exit 1; }
echo "Backing up to ${backup_file}..."
tar -zcf "${backup_dir}/${backup_file" "${source_dir}" "${@}" 2>/dev/null
sync
echo "${today}: backup successful" >> "$logfile"
echo "Done."
```

OR

```
#!/bin/bash

today=$(date +%Y%m%d)
backup_dir=/tmp
backup_file=backup${today}.tgz
source_dir="/root"
logfile=/var/log/backup.log

if [ $(id -u) -ne 0 ] ; then
  echo "Execute as root!" 1>&2
  exit 1
fi
```

```
echo "Backing up to ${backup_file}..."
tar -zcf "${backup_dir}/${backup_file" "${source_dir}" "${@}" 2>/dev/null
sync
echo "${today}: backup successful" >> "$logfile"
echo "Done."
```

Exercise

Write a script `copy2dir` that expects 2 parameters:

- If there are not 2 parameters, an error should be shown and the script stops with an exit code equal to 1
- If the second parameter is not a directory, an error is shown and the script stops again with exit code 1
- If the first parameter is not a file or is not readable, the script stops with exit code 1
- If the parameters are in order, the script copies the file into the directory

Make sure you *test* the script.
And write the script one step at a time: write code, *test*, write more code, *test* ...

Iterations

For-loop

```
#!/bin/bash
for file in *
do
  [ -x "$file" ] && echo "$file is executable"
done
```

While

Example to process all arguments of a script:

```
#!/bin/bash
while [ $# -ne 0 ]
do
  echo "$1"
  shift
done
```

Iterating through lines of a file:

```
#!/bin/bash
while read -r line
do
```

```
  echo "$line"
done < file
```

Until

And to be fully complete, the least used loop statement of the world: `until`.

```
#!/bin/bash
counter=20
until [ "$counter" -lt 10 ]
do
  echo counter "$counter"
  counter=$((counter-1))
done
```

Exercise

Write a script that iterates through all files and directories in the current directory and states whether it is a file or a directory.

Extend the previous script so that you can pass a parameter specifying the directory. Check if this is a directory.

Case

As in other programming languages, you can use a case statement to simplify complex conditionals when there are multiple different choices. So rather than using a few if, and if-else statements, you could use a single case statement.

```
#!/bin/bash
case $1 in
  start) echo "starting service" ;;
  stop) echo "stopping service" ;;
esac
```

Example with another version of the backup script:

```
#!/bin/bash
backup_file=/tmp/etc.tar.gz

[ $(id -u) -eq 0 ] || { echo "run $0 as root" >&2 ; exit 1 ; }
[ -z "$1" ] && { echo "usage: $0 [now] | [restore]" >&2; exit 1 ; }

case "$1" in
  now)
    echo "starting backup..."
```

```
    tar -czf "${backup_file}" -C /etc . 2>/dev/null
    ;;
  restore)
    echo "restoring backup to /tmp/etc..."
    mkdir -p /tmp/etc
    tar -xzf "${backup_file}" -C /tmp/etc
    ;;
  *)
    # this is the default
    echo "usage: $0 [now] | [restore]"
esac
```

Functions

You can also define functions (cf. methods):

```
function memory_check() {
    echo ""
    echo "The current memory usage is: "
    free -h
    echo ""
}
```

Parameters

You don't need to declare parameters, just use `$1`, `$2`, ..., `$9` in functions.
And no use of parentheses when calling the function

```
$ nl param_test.sh
     1  #!/bin/bash
     2  param_test() {
     3      echo "\$1 in funtion: $1"
     4  }
     5  echo "\$1 in main   : $1"
     6  param_test B
$ ./param_test.sh A
$1 in main   : A
$1 in funtion: B
```

One can consider a function as a script within a script:

```
$ nl param_test.sh
     1  #!/bin/bash
     2  param_test() {
     3      echo "\$1 in funtion: $1"
     4  }
     5  echo "\$1 in main   : $1"
```

```
     6  par_test B
$ ./param_test.sh A
$1 in main   : A
./param_test.sh: line 6: par_test: command not found
```

```
#!/bin/bash
today=$(date +%Y%m%d)
backup_dir=/tmp
backup_file=backup${today}.tgz
source_dir="/root"
logfile=/var/log/backup.log

fatalError() {
  echo "$1" 1>&2
  [ -w "${logfile}" ] && echo "${today}: $1" >> "${logfile}"
  exit 1
}

[ $(id -u) -eq 0 ] || fatalError "Execute as root!"
echo "Backing up to ${backup_file}..."
tar -zcf "${backup_dir}/${backup_file" "${source_dir}" "${@}" 2>/dev/null
|| fatalError "Cannot create archive"
sync
echo "${today}: backup successful" >> "$logfile"
echo "Done."
```

Some best practices:

- Function names: lower-case, with underscores to separate words (cf. Google's Shell Style Guide)
- There is also a function keyword, but its use is *not* recommended

  ```
  function some_function() {

  }
  ```bash
  ```

Code review

Do you want to have your code reviewed? Use shellcheck!
Install: `sudo dnf install shellcheck`
Use: `shellcheck script`

Use of `shellcheck` is **strongly recommended**.

Exercises
  ```

Basic exercises

9.1 What does the following script do?

```
#!/bin/bash

# Script assumes 'num_args' for $# and 'input_dir' for $1
if [ \( "$num_args" -ne 1 \) -o \( ! -d "$input_dir" \) ]
then
  echo "Usage: $0 <folder>" 1>&2
fi
```

9.2 Hidden files

Create a script named **"count_hidden.sh"** that counts how many **hidden** folders and files are in a directory.

The directory is passed as a **parameter** to the script.

The script must check if the parameter is present and if it refers to a directory.

Print the following to the screen:

"The directory ... contains ... hidden files."

9.3 backup.sh

Write a script named **"backup.sh"** that:

- **Must be started as root.** Check this; if not, output an error message and stop the script.
- The script creates a gzipped tar backup of the **/var/log** directory, to a **directory you pass as a parameter** to the script. The filename is **"logfiles.tgz"**.
- If no parameter is given, use **/tmp** as the **default** directory.
- Error messages are written to the **logfile /tmp/logfile**.
- At the end of the script, output how many **seconds** the script took to run. (Hint: use `date +%s`)

9.4 install.sh

Write a script named **"install.sh"** that tries to create a directory named `prog` in `/root`. If it succeeds, it copies all files from your home directory with the `.sh` extension to `/root/prog`.

If it fails, no error message should be displayed, and the script will create the directory in `/tmp` and copy the `.sh` files there.

Also, test that no error messages appear if you run the script a second time.

9.5 Large files

Create a script named **"large.sh"** that for all directories in `$PATH`: prints the files that are larger than `x` MB.

`x` is passed as a **parameter**.

9.6 Rescale Images

Knowing that `"convert inputfile -resize 100 outputfile"` rescales an image (inputfile) so that its width becomes 100 pixels, write a script named **"rescale.sh"** that rescales all `.jpg` files in `~/Pictures` to a width of 100 pixels.

The rescaled images must have the same filename, but with a prefix **"scaled_"**.

9.7 Ping

Write a script named **"ping.sh"** that uses a file named **"hosts"** which lists a number of servers.

Create this file first.

For each line in this file, perform a 'ping' (ensure ping stops after 1 request).

The output must be appended to the file **"/tmp/ping_result"**.

9.8 Host monitoring

1. Create a file named **"pinghosts"** that contains the following:

   ```
   www.kdg.be
   this.does.not.exist
   www.google.be
   ```

   Write a script named **"pinghosts.sh"** that pings the hosts in this file one by one (once, see `man ping`). The next host is pinged every second, and this continues in an infinite loop. No output should appear on the screen, but write the following to **"/tmp/loghosts"**:

   ```
   Mon Dec 3 07:52:26 CET 2018;www.kdg.be;OK
   Mon Dec 3 07:52:27 CET 2018;this.does.not.exist;NOK
   ```

2. Write a script named **"ping_stats.sh"** that counts how many "OK" and how many "NOK" entries appear in **"/tmp/loghosts"** and outputs the following:

   ```
   Ping statistics:
   OK: 186
   NOK: 93
   ```

9.9 Website KdG monitoring

Create a script named **"kdg_up.sh"** that checks every minute whether the website `www.kdg.be` is online. A ping to the site is not sufficient. Check if the KdG start page contains the string "Karel de Grote Hogeschool".

If you receive no response from the site, write the current time to a logfile defined in a variable.

Wait one minute between checks using: `"sleep 60"`.

To retrieve the web page, you can use the command `"wget -O- http://www.kdg.be"` or `"curl -s https://www.kdg.be"`.

9.10 Animation

Create a script named **"animation.sh"**. Add the following lines:

```
#!/bin/bash
echo -n "/"
sleep 1
```

Now ensure that the cursor remains in the same location and that the following characters appear successively: `/-\|/-\|` (this creates a rotating effect).

Hint: use the backspace character (`\b`)

Create a loop that shows this animation for a number of seconds. The number of seconds is configurable with a **parameter**.